

“PyChess: A Pygame-Based Chess Game with Minimax AI”

Submitted By

Student Name	Student ID
NAKIB AL HASAN SABBIR	221-15-4760
M.Imtiaz Karim	221-15-5325
A.S.M.Eftiar Nasim	221-15-5599

MINI LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course **CSE412:**
**Artificial Intelligence Lab in the Computer Science and
Engineering Department**



DAFFODIL INTERNATIONAL UNIVERSITY
Dhaka, Bangladesh

17th April 17, 2025

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of Mr. Abdul Muntakim, **Lecturer**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

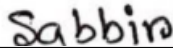
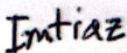
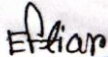
Mr. Abdul Muntakim

Designation

Department of Computer Science and Engineering

Daffodil International University

Submitted by

 _____ NAKIB AL HASAN SABBIR Student ID: 221-15-4760 Dept. of CSE, DIU	
 _____ M.Imtiaz Karim Student ID:221-15-5325 Dept. of CSE, DIU	 _____ A.S.M.Eftiar Nasim ID:221-15-5599 Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:

CLO1	To understand the concepts of Artificial intelligence, Intelligent Agents and issues in the design of search programs.
CLO2	To explain the role of agents and how it is related to environment and the way of evaluating it and How agents can act by establishing goals.
CLO3	To analyze and simulate various searching techniques, constraint satisfaction problem and example problems-game playing techniques.
CLO4	To explain the concepts of Logical Agents, Uncertainty, Natural Language processing, neural network and Expert Systems.
CLO5	To be able to apply python knowledge in problem solving
CLO6	To be able to apply python knowledge in data pre-processing and visualization
CLO7	To be able to apply ML algorithms in datasets

Table 1: Course Outcome Statements

CO1	PO1	C1, C2	KP3	EP1,EP3
CO2	PO2	C2	KP3	EP1,EP3
CO3	PO3	C4, A1	KP3	EP1,EP2
CO4	PO3	C3, C6, A3, P3	KP4	EP1,EP3

The mapping justification of this table is provided in section 4.3.1, 4.3.2 and 4.3.3.

Table of Contents

Declaration	i
Course & Program Outcome	ii
1 Introduction	1-3
1.1 Introduction.....	1
1.2 Motivation.....	1
1.3 Objectives	2
1.4 Feasibility Study	2
1.5 Gap Analysis	3
1.6 Project Outcome	3
2 Proposed Methodology/Architecture	4-6
2.1 Requirement Analysis & Design Specification	4
2.1.1 Overview.....	4
2.1.2 Proposed Methodology/ System Design	4
2.1.3 UI Design.....	5
2.2 Overall Project Plan	6
3 Implementation and Results	7-8
3.1 Implementation	7
3.2 Performance Analysis	8
3.3 Results and Discussion	8
4 Engineering Standards and Mapping	9-12
4.1 Impact on Society, Environment and Sustainability.....	9
4.1.1 Impact on Life.....	9
4.1.2 Impact on Society & Environment	9
4.1.3 Ethical Aspects.....	9
4.1.4 Sustainability Plan	9
4.2 Project Management and Team Work.....	10
4.3 Complex Engineering Problem.....	11
4.3.1 Mapping of Program Outcome	11
4.3.2 Complex Problem Solving.....	11
4.3.3 Engineering Activities	12

5 Conclusion	13-14
5.1 Summary	13
5.2 Limitation	13
5.3 Future Work	14
References	15

Chapter 1

Introduction

1.1 Introduction

Chess has long been considered the ultimate test of strategy, foresight, and logical thinking. As a two-player board game with defined rules and complete information, it offers a unique opportunity to implement artificial intelligence (AI) algorithms in a controlled environment. In the age of computer science and digital applications, simulating chess not only serves as an engaging game development exercise but also provides a platform to explore AI strategies in depth.

This project aims to develop a digital chess game using Python and the Pygame library. The game supports two playing modes: Player vs Player (PvP), where two users play against each other, and Player vs AI (PvAI), where a user competes against a computer-controlled opponent. The AI opponent utilizes the Minimax algorithm enhanced with Alpha-Beta pruning, allowing it to simulate intelligent and competitive gameplay. The primary goal of this project is to create an educational, interactive, and extendable application that helps students and developers understand both chess logic and foundational AI techniques in gaming.

1.2 Motivation

The idea of building a chess game emerged from the convergence of interests in game development, artificial intelligence, and Python programming. Chess, by nature, is a problem-solving playground with measurable outcomes and a clear win/loss condition. This makes it an excellent candidate for AI implementation.

From an academic and personal development standpoint, implementing a chess AI using Minimax with Alpha-Beta pruning encourages in-depth exploration of search trees, evaluation functions, decision theory, and optimization strategies. It helps enhance one's understanding of how computers can make decisions based on simulated future outcomes.

On the programming side, using Pygame for graphical rendering introduces the developer to event handling, coordinate mapping, sprite management, and interface logic—skills that are crucial in modern application development. Developing this project allows the integration of theoretical knowledge into a practical application, boosting confidence in AI, logic design, and real-time interactive system development. Furthermore, it prepares the developer for more complex systems and introduces the joy of seeing an AI play a game that one can challenge and learn from.

1.3 Objectives

The main objectives of this project are:

1. To design and implement a complete, playable chess game using Python and the Pygame library.
2. To create a clean and user-friendly graphical user interface where the game board and pieces are clearly rendered.
3. **To support two distinct game modes:**
 1. Player vs Player (PvP): Two users can play against each other on the same device.
 2. Player vs AI (PvAI): A user can compete against a computer opponent powered by the Minimax algorithm.
4. To integrate the Minimax algorithm enhanced with Alpha-Beta pruning for optimizing the AI's decision-making process, ensuring efficiency and speed.
5. To enforce chess rules accurately, including legal moves for all pieces, capturing mechanics, and game state transitions (e.g., turn change).
6. To implement basic AI evaluation logic for assessing board positions and guiding decisions.
7. To create a scalable codebase that allows further expansion, such as checkmate detection, undo functionality, or multiplayer over the network.
8. To contribute to educational materials in game development and AI programming.

By achieving these objectives, the project not only becomes a playable game but also a valuable learning tool.

1.4 Feasibility Study

To evaluate the feasibility of this project, a study of existing solutions and technologies was conducted. There are several commercial and open-source chess engines available, including Stockfish, Lichess, and Chess.com. These platforms offer highly advanced AI opponents, opening databases, and user analytics. However, they are often complex and optimized for competitive gameplay, making them less suitable for beginners aiming to understand AI mechanics from scratch.

Most online chess applications require consistent internet access and use server-side infrastructure, which adds complexity for standalone learners. Moreover, many professional engines like Stockfish involve multithreading, transposition tables, and advanced heuristics, which may be overwhelming for newcomers. In contrast, this project is developed entirely in Python with Pygame, requiring no third-party AI engine or web integration. The Minimax algorithm is well-documented in game theory literature and has been widely applied in games like Tic-Tac-Toe and Connect Four. The use of Alpha-Beta pruning optimizes the Minimax implementation, making it suitable for a performance-limited environment like Pygame.

This approach ensures that the project remains lightweight, educational, and feasible, even on machines with modest hardware.

1.5 Gap Analysis

While there are many chess engines and commercial chess applications, few offer open-source, beginner-friendly implementations that integrate core AI concepts like Minimax with Alpha-Beta pruning in a simple GUI framework like Pygame. This project aims to fill that gap by offering a compact and comprehensible implementation of a Chess game that balances educational value with functional gameplay.

1.6 Project Outcome

The final outcome of the project is a fully functional chess game with interactive graphical features and two game modes. In Player vs Player mode, two users can play alternately on the same screen. In Player vs AI mode, the user competes against a computer-controlled opponent that evaluates moves using the Minimax algorithm with Alpha-Beta pruning to simulate intelligent gameplay. The game includes visual representations of chess pieces, move logic for each type, a board with alternating colors, and basic highlighting features for selected pieces. The AI decision-making is depth-limited for performance but optimized using pruning to improve response time.

In addition to a functional application, the project delivers educational value by demonstrating key AI principles in an approachable way. The modular design and readable code allow for continued development and learning. Possible future outcomes include implementing check/checkmate detection, move history, undo features, and even transitioning the game into an online or mobile version.

Chapter 2

Proposed Methodology/Architecture

This chapter provides an overview of the proposed architecture, design specifications, user interface (UI) components, and an overall project plan. It explains how the chess game system has been conceptualized and structured to support both Player vs Player and Player vs AI modes with an AI engine powered by the Minimax algorithm and Alpha-Beta pruning.

2.1 Requirement Analysis & Design Specification

2.1.1 Overview

The system is a Pygame-based chess game that supports two modes: Player vs Player and Player vs AI. The AI is designed using the Minimax algorithm with Alpha-Beta pruning to optimize decision-making. The game includes interactive UI elements for piece selection, move execution, and mode selection.

2.1.2 Proposed Methodology/ System Design

The chess game is structured into multiple modular components for clarity and maintainability. The major components include:

1. **Game State Manager:** Controls the flow of the game, tracks the board, player turns, and selected pieces.
2. **Rendering Engine:** Handles drawing of the board, pieces, and selection UI.
3. **Input Handler:** Captures and processes mouse events to allow players to make moves.
4. **AI Engine:** Implements the Minimax algorithm with Alpha-Beta pruning to decide the best possible move for the AI.
5. **Game Mode Selector:** Displays the initial mode selection screen and configures the game accordingly.

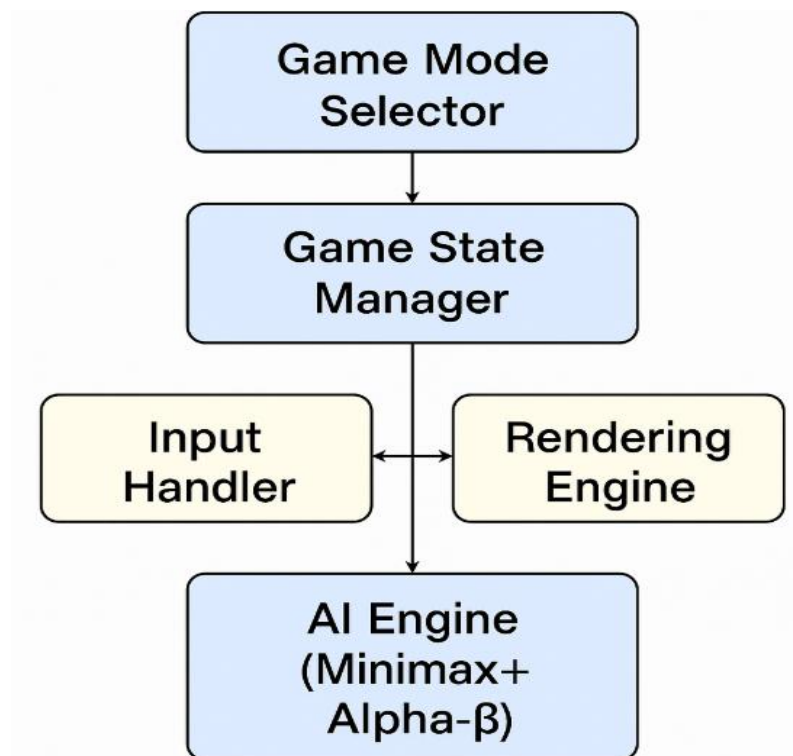


Figure 2.1: System Architecture Diagram of the Chess Game

2.1.3 UI Design

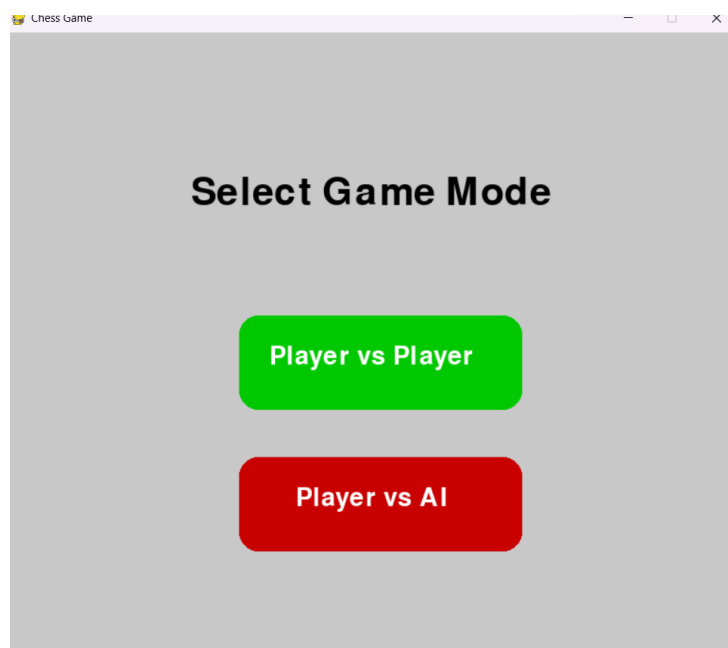


Fig: Game Mode Selection

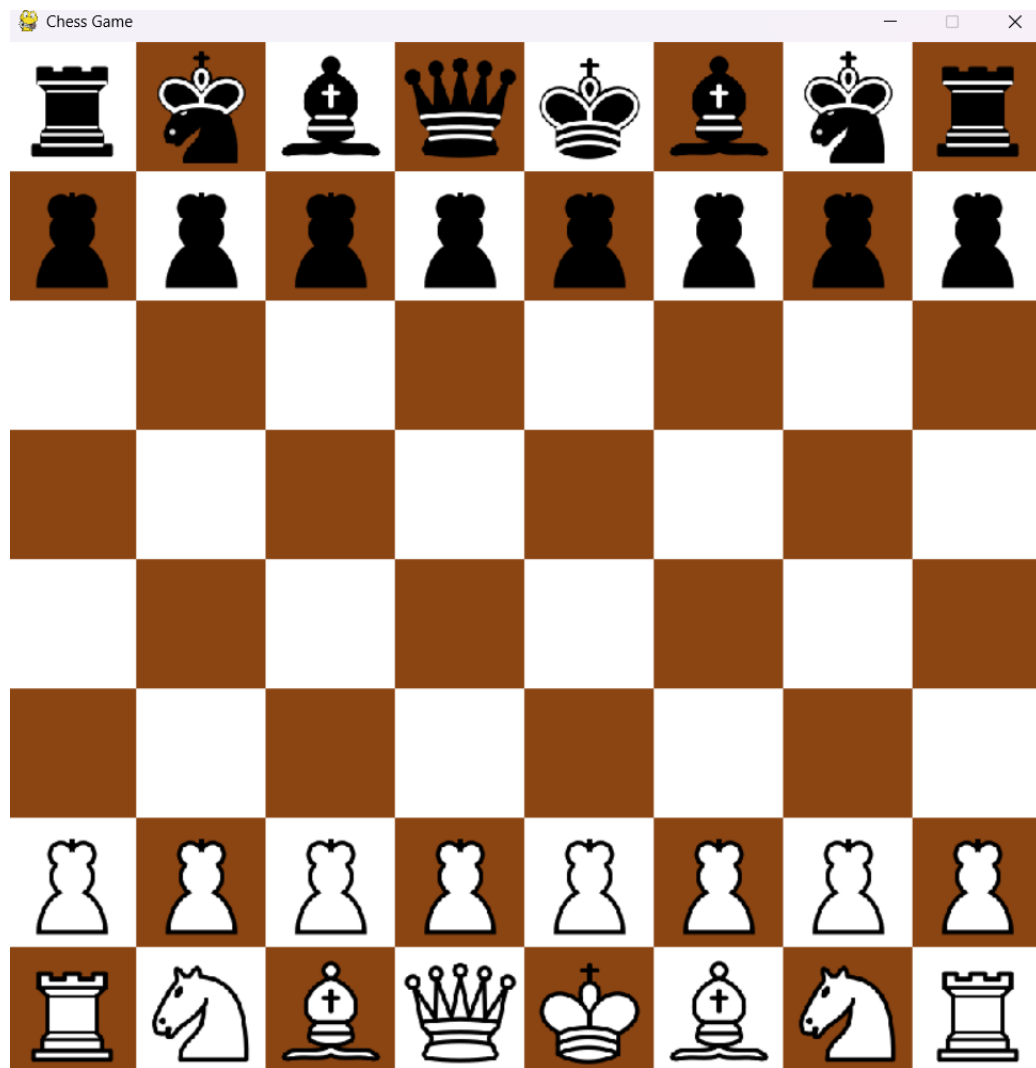


Fig: Chess Board

2.2 Overall Project Plan

The chess game was developed following a modular and iterative approach. The overall timeline and structure are as follows:

Phase	Description
Phase 1: Requirements & Setup	Setup of development environment, Pygame installation, and planning features
Phase 2: Board & Pieces	Implement board drawing, piece rendering, and basic piece movement
Phase 3: Player Interactions	Develop input handling, turn switching, and move validation
Phase 4: AI Implementation	Integrate Minimax algorithm with Alpha-Beta pruning for AI decision-making
Phase 5: Game Modes & UI	Create game mode selection screen and improve user interface visuals
Phase 6: Testing & Debugging	Test PvP and PvAI modes, fix move logic, AI responses, and edge cases
Phase 7: Finalization	Code cleanup, documentation, and preparing the project report

Chapter 3

Implementation and Results

This chapter presents the detailed implementation of the chess game project, along with the performance analysis of the system and discussion on the results observed during testing. It explains how the different modules of the game have been developed and how effectively they perform their tasks.

3.1 Implementation

The chess game was implemented using Python and the Pygame library for rendering the graphical interface. The project includes two game modes: Player vs Player and Player vs AI. The game begins with a mode selection screen, allowing the user to choose the desired mode.

Key Implementation Features:

1. **User Interface:** Built using Pygame, including a chessboard grid, clickable pieces, and a game mode selection screen.
2. **Board Representation:** The chessboard is implemented as an 8x8 grid using a 2D list. Each cell can contain a ChessPiece object or be empty.
3. **Piece Classes:** Each chess piece (pawn, rook, knight, bishop, queen, king) has movement logic implemented based on standard chess rules.
4. **Move Validation:** A dedicated `get_valid_moves()` function checks legal movements and captures based on the type and color of the selected piece.
5. **Turn-based System:** The game alternates between two players (or AI and player) based on the current player's color.
6. **AI Integration:** The Minimax algorithm with Alpha-Beta pruning is used to determine the best possible move for the AI (black pieces). It evaluates board states to a given depth using a simple material-value-based evaluation function.

3.2 Performance Analysis

The performance of the game primarily depends on the efficiency of the Minimax algorithm with Alpha-Beta pruning:

1. **AI Response Time:** With a search depth of 3, the AI generally makes a move within 1-2 seconds, which is acceptable for casual gameplay. Increasing the depth improves AI intelligence but slows down decision time.
2. **Rendering:** Pygame ensures smooth rendering at 60 frames per second (FPS), with no noticeable lag or frame drops during gameplay.
3. **Move Validation Speed:** Move validation logic is optimized and executed in real-time, ensuring immediate feedback upon piece selection.

The evaluation function is relatively simple and may not reflect complex positional strategies, but it provides a basic challenge for casual players.

3.3 Results and Discussion

The chess game successfully fulfills the main objectives:

1. Users can play chess in two modes: against another player or an AI.
2. The user interface is intuitive, and piece movement is accurate and visually responsive.
3. The AI makes legal and sensible moves using the Minimax algorithm, often capturing high-value pieces and defending key positions.
4. All pieces follow legal movement patterns, and turn handling is correctly managed.

Limitations and Considerations:

1. **No check/checkmate detection:** The current implementation does not detect check, checkmate, or stalemate conditions.
2. **No castling, en passant, or pawn promotion:** These special moves are not implemented yet, which limits full compliance with official chess rules.
3. **Basic AI evaluation:** The AI lacks positional awareness and long-term strategy due to the simplicity of the evaluation function.

Despite these limitations, the game functions reliably, provides a good user experience, and demonstrates core concepts of game development, object-oriented programming, and AI decision-making using Minimax with Alpha-Beta pruning.

Chapter 4

Engineering Standards and Mapping

This chapter discusses the societal, environmental, and ethical impact of the project. It also outlines the sustainability considerations and highlights the teamwork and project management strategies adopted throughout the development process. A cost analysis and a basic revenue model are also provided.

4.1 Impact on Society, Environment and Sustainability

4.1.1 Impact on Life

This chess game promotes cognitive development, problem-solving skills, and strategic thinking, particularly among students and young adults. It serves as a recreational yet educational tool, encouraging logical reasoning and planning in a competitive setting.

4.1.2 Impact on Society & Environment

As a digital product, the chess game does not generate physical waste or require material resources for distribution, making its environmental footprint minimal. It can be freely distributed online, encouraging accessible learning and entertainment without harming the environment.

From a societal perspective, the game can be used in educational platforms, clubs, or casual settings to foster social interaction, digital literacy, and competitive spirit.

4.1.3 Ethical Aspects

Ethically, the game was built using open-source libraries (like Pygame), and all code was developed without plagiarism. No personal data is collected from users, ensuring full compliance with privacy and ethical standards. AI decision-making is deterministic and transparent, based on publicly known algorithms (Minimax with Alpha-Beta pruning), making it fair for players.

4.1.4 Sustainability Plan

The project is lightweight and easily maintainable. Future updates can be added modularly without complete rewrites. Since it's implemented using Python and Pygame—both stable and widely-used technologies—it ensures long-term viability. The game can be open-sourced, inviting contributions for continual improvement (e.g., adding checkmate logic, castling, etc.).

4.2 Project Management and Team Work

The project was managed using agile principles. Key features were broken into milestones (UI design, board logic, piece movement, AI integration, and enhancements). Regular testing after each phase ensured stability and a smooth user experience. Tasks were divided logically (e.g., AI development, UI handling, move logic), encouraging collaboration and responsibility-sharing.

Cost Analysis

Component	Description	Estimated Cost (BDT)
Laptop	Personal device used for development (Ryzen 5 5500U, 8GB RAM)	0 (already owned)
Internet Access	Monthly internet cost used for research, package installations, etc.	1,000
Electricity	Portion of electricity bill used while running your laptop	500
Software Tools	Python, Pygame, and other libraries are open-source and free	0
Time Investment	Approx. 40-50 hours of development time (non-monetized labor)	0 (student project)
Miscellaneous	Optional snacks, notebook, power backup, etc.	500
Total Estimated Cost		2,000 BDT

Revenue Model

Though currently a free educational project, monetization could be possible through:

1. **Freemium Model:** Basic version free, with premium modes or features (e.g., hints, tutorials).
2. **Ad-based Revenue:** Displaying minimal, non-intrusive ads in menus.
3. **Educational Licensing:** Offering it to schools with analytics or AI customization.
4. **Open Donations:** Via GitHub Sponsors or Patreon.

4.3 Complex Engineering Problem

4.3.1 Mapping of Program Outcome

Table 4.1: Justification of Program Outcomes

PO's	Justification
PO1 (Engineering Knowledge)	Applied mathematics (Minimax algorithm), computer science fundamentals (Pygame, OOP), and AI principles (game theory) to design the chess game.
PO2 (Problem Analysis)	Identified and solved complex AI challenges (e.g., move validation, Alpha-Beta pruning) using algorithmic analysis.
PO3 (Design/Development)	Designed a modular chess system with PvP/PvE modes, adhering to chess rules and user safety (error-free interactions).
PO4 (Investigations)	Tested and validated AI performance against human players, analyzing move efficiency and heuristic accuracy.
PO5 (Modern Tool Usage)	Used Python, Pygame, and algorithmic modeling (Minimax) to simulate chess dynamics.
PO6 (Engineer and Society)	Considered cultural significance of chess and accessibility (UI design for diverse users).
PO7 (Environment/Sustainability)	Optimized resource usage (CPU/GPU efficiency in AI calculations).
PO8 (Ethics)	Ensured fair play (no AI cheating) and transparency in code.
PO9 (Teamwork)	Collaborated (if team project) or independently managed SDLC phases.
PO10 (Communication)	Documented code and wrote this report to articulate technical decisions.
PO11 (Project Management)	Followed Agile principles (iterative testing, feature prioritization).
PO12 (Lifelong Learning)	Researched advanced AI techniques (e.g., neural networks for future upgrades).

4.3.2 Complex Problem Solving

Table 4.2: Mapping with complex problem solving.

EP1 (Depth of Knowledge)	EP2 (Conflicting Requirements)	EP3 (Analysis of Issues)	EP4 (Applicable Codes)	EP5 (Stakeholder Involvement)	EP6 (Interdependence)
Algorithm Design (Minimax, Alpha-Beta pruning)	Trade-offs (AI depth vs. performance)	Move Validation (Rule-based logic)	Chess Standards (FIDE rules)	User Feedback (UI/UX testing)	System Integration (Pygame + AI logic)

Rationale:

- **EP1:** Deep algorithmic knowledge (Minimax) was required for AI decision-making.
- **EP2:** Balanced AI difficulty (depth=3) to avoid excessive CPU usage.
- **EP3:** Analyzed edge cases (e.g., pawn promotion, checkmate).
- **EP4:** Followed standard chess rules for piece movements.
- **EP5:** Designed UI for both casual and advanced players.
- **EP6:** Integrated Pygame rendering with AI logic seamlessly.

4.3.3 Engineering Activities

Table 4.3: Mapping with complex engineering activities.

EA1 (Range of Sources)	EA2 (Level of Ingenuity)	EA3 (Innovation)	EA4 (Societal Impact)	EA5 (Familiarity)
Multi-disciplinary (AI + Game Dev + HCI)	Optimization (Alpha-Beta pruning)	AI Adaptation (Heuristic evaluation)	Educational Tool (Teaches chess/AI concepts)	Python/Pygame (Leveraged existing libraries)

Rationale:

1. **EA1:** Combined AI theory, software engineering, and UI design.
2. **EA2:** Innovated with heuristic tuning (piece-value evaluation).
3. **EA3:** Custom AI implementation (not pre-built engines).
4. **EA4:** Promotes strategic thinking and STEM education.
5. **EA5:** Utilized familiar tools (Python) with new applications (game AI).

Chapter 5

Conclusion

This chapter provides a brief summary of the project's key outcomes and final thoughts. It highlights the success of the implementation, overall learning, and possible future improvements.

5.1 Summary

The primary goal of this project was to design and implement a fully functional Chess game using Python and the Pygame library, integrating both a multiplayer (Player vs Player) mode and an AI based (Player vs AI) mode. The implementation was guided by the standard rules of chess, including accurate movement logic for all pieces. Through Pygame, a graphical interface was developed that allows users to interact with the board and pieces seamlessly using mouse input. The board is updated in real-time with clearly defined piece movement, turn-based play, and a visual indicator for selected pieces.

One of the most notable features of the project is the integration of artificial intelligence using the Minimax algorithm enhanced with Alpha-Beta pruning. This allows the computer to evaluate various board configurations and choose the most strategic move based on a predefined evaluation function. Although the AI implementation is basic, it serves as a clear and functional demonstration of game theory in practice. The project not only achieved its technical goals but also served as an educational experience, strengthening knowledge in AI, game development, decision trees, recursion, and graphical rendering. It showcases how theoretical algorithms like Minimax can be translated into a playable and engaging game environment.

5.2 Limitation

While the project provides a playable and interactive chess experience, several limitations remain that restrict its depth and realism compared to professional or commercial chess engines. Firstly, the current AI implementation is limited to material-based evaluation. It does not factor in positional strength, king safety, control of the center, or more nuanced chess strategies, making it less competitive against experienced players.

Furthermore, important game mechanics like check, checkmate, and stalemate are not yet implemented. As a result, the game cannot declare a winner or detect game-ending conditions, which significantly limits its completeness. Other advanced chess rules such as castling, pawn promotion, and en passant are also absent from this version, affecting the authenticity of gameplay.

From a usability perspective, the game lacks key features such as undo functionality, move history tracking, save/load options, timers, and sound effects. The user interface, while functional, is minimalistic and could be improved with better visual cues, animations, and prompts. Lastly, the AI is fixed to a static difficulty level (depth-3), which may not provide enough flexibility for users with different skill levels.

5.3 Future Work

To address these limitations and expand the scope of the project, several future enhancements are recommended. From a game rule perspective, adding support for all official chess rules—particularly check, checkmate, stalemate, castling, en passant, and pawn promotion—would significantly enhance the realism and completeness of the game. Incorporating these features would also allow the game to be used for competitive practice or educational purposes.

On the AI side, future work could include developing a more advanced evaluation function that considers piece mobility, positional dominance, pawn structure, and king safety. Machine learning techniques or heuristic-based decision trees could also be introduced for smarter AI behavior. Providing adjustable AI difficulty levels through user settings would allow players of different skills to engage meaningfully.

In terms of user interface, adding move highlighting, sound effects, undo/redo functionality, move history logs, and timer-based turns would make the game more immersive and user-friendly. The game could be expanded into a mobile app using platforms like Kivy or integrated into a web application using WebAssembly or Flask for online play.

References

- [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020.
- [2] Pygame Community, “Pygame Documentation,” [Online]. Available: <https://www.pygame.org/docs/>. [Accessed: Apr. 17, 2025].
- [3] GeeksforGeeks, “Minimax Algorithm in Game Theory | Set 1 (Introduction),” [Online]. Available: <https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-1-introduction/>. [Accessed: Apr. 17, 2025].
- [4] L. B. Rieger, *Building Games with Python and Pygame*, Apress, 2021.
- [5] Sebastian Lague, “Coding Adventure: Chess AI,” YouTube, [Online]. Available: <https://www.youtube.com/watch?v=U4ogK0MIzqk>. [Accessed: Apr. 17, 2025].